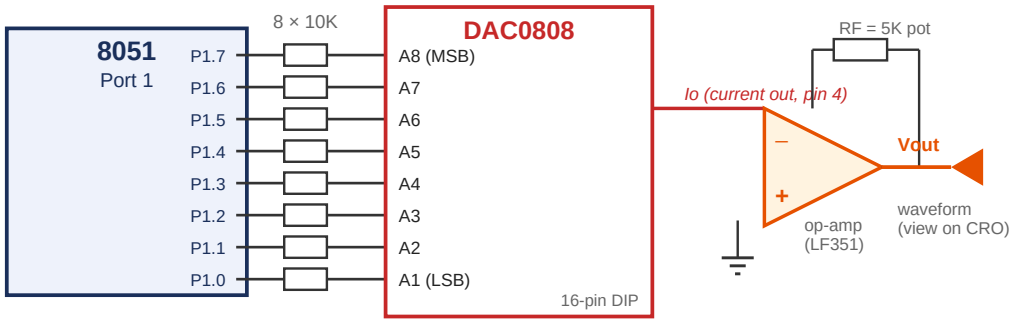
What the exam expects: DAC role • DAC0808 connections • and for EACH waveform (square, pulsed, triangular, sawtooth, sine): the **logic**, the **pin pattern on P1** and the **simplified program**. **How to read:** every wave gets a picture with numbered markers (1)–(5), a walkthrough table and a code column — the numbers are the SAME in all three.

1 • Idea — why a DAC makes waveforms

- DAC0808 = **8-bit Digital-to-Analog Converter**: an 8-bit digital input → an **analog output current** (larger number → larger current).
- The op-amp (I–V converter) turns the current into a voltage: **Vout = –Io × RF** — so **Vout ∝ the value written on P1**.
- So a waveform is made by **writing a chosen sequence of numbers to P1 with delays** — the sequence decides the SHAPE, the delay decides the FREQUENCY.
- Every wave program has the same skeleton: **set value → MOV P1,A → ACALL DELAY → change value → repeat (SJMP)**.

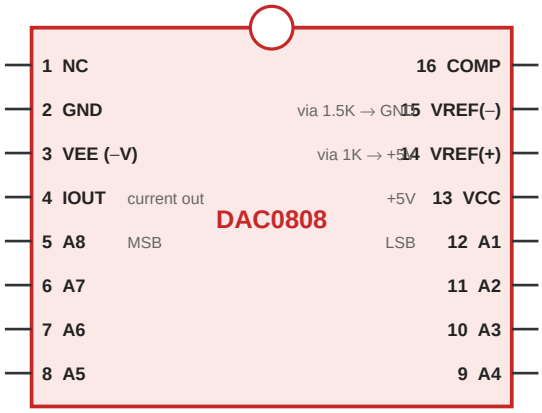


VCC 13 = +5V • VEE 3 = –5V • GND 2 • VREF(+) 14 → 1K → +5V • VREF(–) 15 → 1.5K → GND • COMP 16 → 0.1µF

Digital value on P1 → DAC current Io → op-amp I–V converter → Vout = –Io × RF

SAME circuit for all five waves — only the pattern written on P1 changes

2 • DAC0808 pin diagram (16-pin DIP)

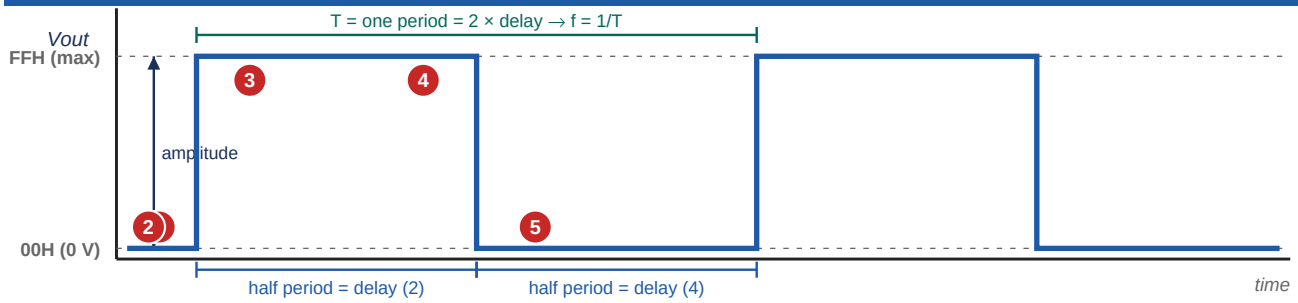


16-pin DIP • numbering:
left 1↓8 top→bottom,
right 9↓16 bottom→top

A1–A8 = 8 digital inputs (LSB→MSB)

Pin roles: A1–A8 = digital inputs (A8 = MSB → connect to P1.7) • VREF(+) sets the reference current • IOUT = analog current out • COMP = compensation capacitor • VCC +5 V, VEE –5 V. Delay subroutine (shared by every program below): `MOV R6,#N / MOV R7,#255 / DJNZ R7 / DJNZ R6 / RET`.

3 • SQUARE WAVE (slides 13 & 18)



only 2 levels: 00H ↔ FFH • equal delays = 50% duty cycle

Logic: only two levels 00H / FFH • equal delays → 50% duty • T = 2 × delay

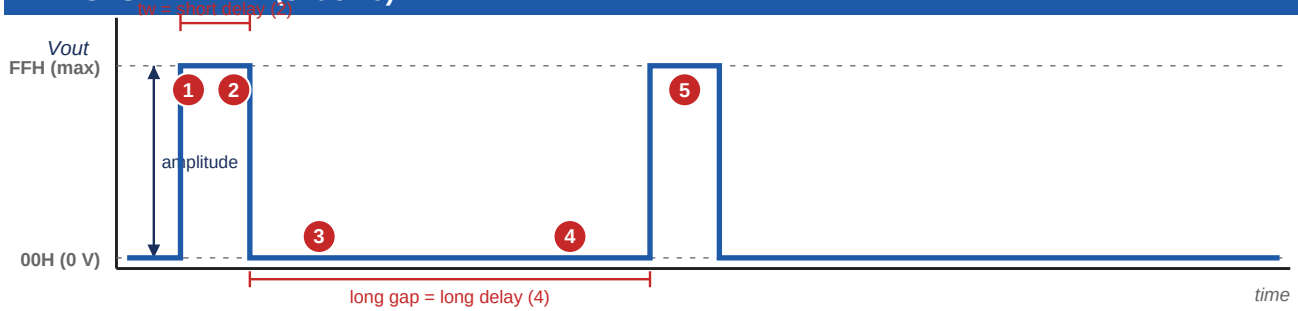
Walkthrough — follow one cycle: (numbers match the ①…⑤ on the wave and in the code)

#	Code does	P1 / A	What you see
1	MOV P1,#00H	`00H = 0000 0000`	Vout drops to 0 V — LOW half begins
2	ACALL DELAY	–	LOW is held → first half period
3	MOV P1,#0FFH	`FFH = 1111 1111`	Vout jumps to max — HIGH half begins
4	ACALL DELAY	–	HIGH held → one full period T is complete
5	SJMP MAIN	–	jumps back → next period, wave repeats

The code — same (1)…(5) as the wave above

```
MAIN:  MOV P1,#00H ; (1) LOW = 0 V
       ACALL DELAY ; (2) hold = half T
       MOV P1,#0FFH ; (3) HIGH = max V
       ACALL DELAY ; (4) hold = half T
       SJMP MAIN ; (5) repeat
```

4 • PULSED WAVE (slide 19)



same 2 levels as square but delays UNEQUAL • T = tw + gap • two delay routines

Logic: same 2 levels, delays unequal • short HIGH pulse, long LOW gap • T = tw + gap

Walkthrough — follow one cycle: (numbers match the ①…⑤ on the wave and in the code)

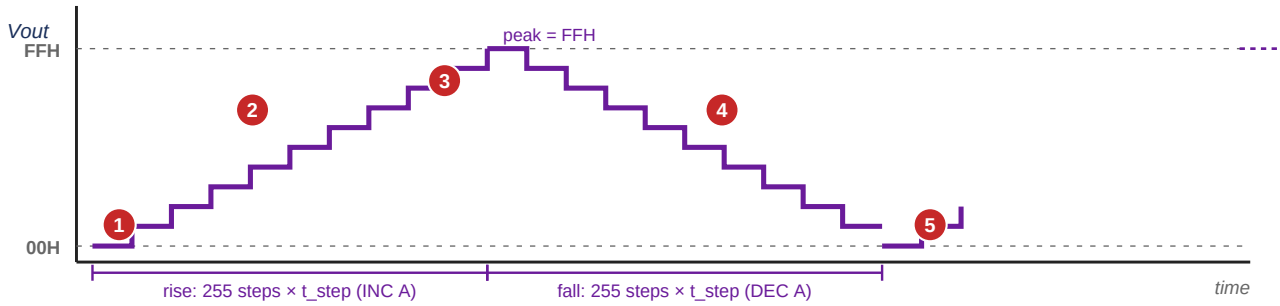
#	Code does	P1 / A	What you see
1	MOV P1,#0FFH	`FFH = 1111 1111`	output rises → pulse starts
2	ACALL DSHORT	–	tiny count → narrow width tw
3	MOV P1,#00H	`00H = 0000 0000`	output falls back to 0 V
4	ACALL DLONG	–	big count → long flat gap
5	SJMP MAIN	–	next pulse; period T = tw + gap

The code — same (1)…(5) as the wave above

```
MAIN:  MOV P1,#0FFH ; (1) pulse goes HIGH
       ACALL DSHORT ; (2) small count → tw
       MOV P1,#00H ; (3) back to 0 V
       ACALL DLONG ; (4) BIG → long gap
       SJMP MAIN ; (5) repeat → pulse train
```

5 • TRIANGULAR WAVE (slide 20)

T = rise + fall ≈ 510 × t_step • both ramps equal → symmetric triangle



Logic: rise = INC A staircase • fall = DEC A staircase • T ≈ 510 × t_step

Walkthrough — follow one cycle: (numbers match the ①…⑤ on the wave and in the code)

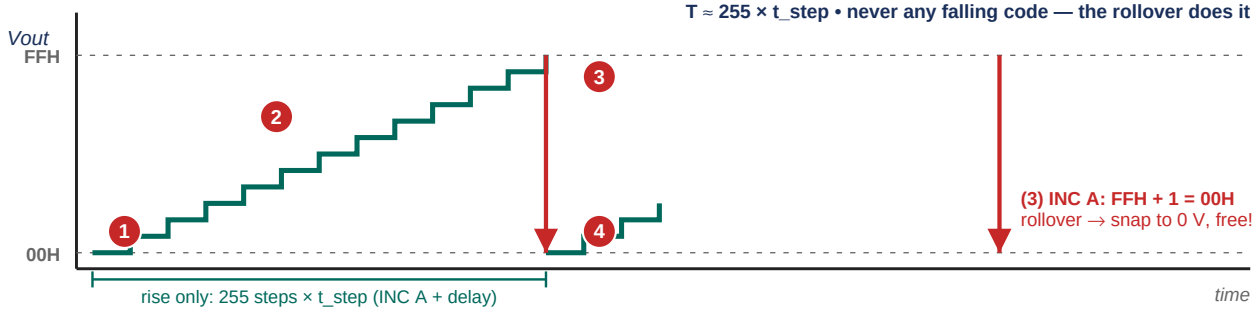
#	Code does	P1 / A	What you see
1	MOV A,#00H	A = 00H	wave starts at 0 V
2	MOV P1,A, DELAY, INC A, JNZ UP	00H→01H→02H →…	staircase rises one step each loop → ramp up
3	JNZ exits when A = FFH	A = FFH = 1111 1111	peak = Vmax reached
4	MOV P1,A, DELAY, DEC A, JNZ DOWN	FFH→FEH→…→01H	staircase falls → symmetric ramp down
5	SJMP MAIN	–	repeat → continuous triangle

The code — same (1)…(5) as the wave above

```
        MOV A,#00H ; (1) start at 0 V
UP:     MOV P1,A ; (2) rising ramp
        ACALL DELAY ; step time t_step
        INC A ; 01H, 02H, …
        JNZ UP ; … until FFH
        MOV A,#0FFH ; (3) peak → turn point
DOWN:   MOV P1,A ; (4) falling ramp
        ACALL DELAY
        DEC A ; FEH, FDH, …
        JNZ DOWN ; … until 01H
        SJMP MAIN ; (5) repeat forever
```

6 • SAWTOOTH WAVE (slide 21)

T ≈ 255 × t_step • never any falling code — the rollover does it



Logic: rising ramp only • FFH + 1 rolls over to 00H • T ≈ 255 × t_step

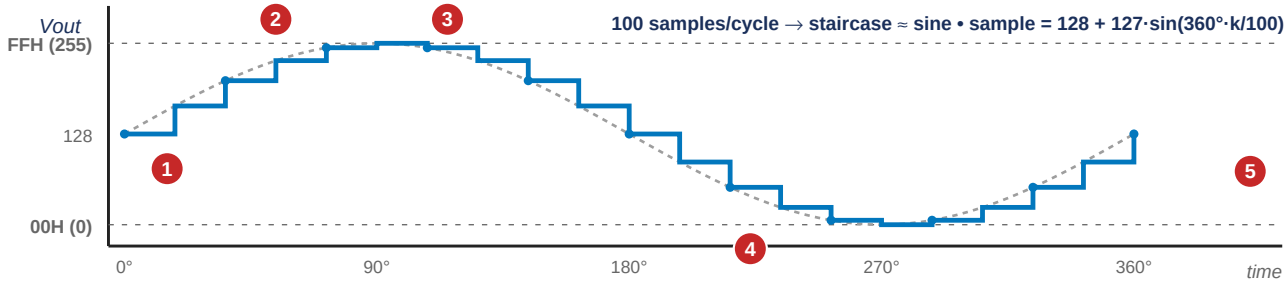
Walkthrough — follow one cycle: (numbers match the ①…⑤ on the wave and in the code)

#	Code does	P1 / A	What you see
1	MOV A,#00H	A = 00H	tooth starts at 0 V
2	MOV P1,A, DELAY, INC A	00H, 01H, 02H, …	slow staircase rise → the tooth
3	INC A when A = FFH	FFH → 00H (!)	carry is dropped → Vout snaps to 0 V (vertical fall, no code)
4	SJMP UP	–	next tooth → sawtooth repeats

The code — same (1)…(5) as the wave above

```
UP:     MOV A,#00H ; (1) tooth starts at 0 V
        MOV P1,A ; (2) send digital step
        ACALL DELAY ; t_step sets frequency
        INC A ; (3) FFH + 1 = 00H rollover
        SJMP UP ; (4) repeat → teeth
```

7 • SINE WAVE (slides 16–17 — lookup-table method)



Logic: can't INC/DEC a sine → table • 100 samples of 128 + 127·sin • T = 100 × t_sample

Walkthrough — follow one cycle: (numbers match the ①…⑤ on the wave and in the code)

#	Code does	P1 / A	What you see
1	MOV DPTR,#TAB	–	DPTR points to first sample in ROM
2	MOVC A,@A+DPTR	`A = 80H (128)`	sample k read from the table (start = mid!)
3	MOV P1,A	`P1 = 80H`	DAC outputs one staircase step
4	INC DPTR … CJNE A,#64H	`88H→FFH→…→00H`	all 100 samples (64H) = one full cycle
5	SJMP MAIN	–	DPTR reset → next cycle repeats

The code — same (1)…(5) as the wave above

```
MAIN:  MOV DPTR,#TAB ; (1) point to table
NXT:   CLR A
       MOVC A,@A+DPTR ; (2) read sample
       MOV P1,A ; (3) sample → DAC step
       ACALL DELAY ; t_sample spacing
       INC DPTR ; (4) next sample
       MOV A,DPL
       CJNE A,#64H,NXT ; 64H = 100 done?
       SJMP MAIN ; (5) restart cycle
TAB:   DB 128,136,144,152 ; 128 + 127·sin t
       DB 160,206,…,255 ; … 100 values
```

8 • Where the table values come from (1 mark favourite)

- Each sample = **128 + 127 × sin(360° × k / 100)**, rounded to the nearest integer (one full cycle in 100 steps).

k (sample)	angle	sin	value 128 + 127·sin	hex written to P1
0	0°	0.00	128	80H = 1000 0000
25	90°	+1.00	255 (peak)	FFH = 1111 1111
50	180°	0.00	128 (mid)	80H = 1000 0000
75	270°	–1.00	0 (trough)	00H = 0000 0000

Why a staircase looks smooth: the DAC holds each sample for t_sample; with 100 samples/cycle the eye (and a filter) sees a sine. More samples / more bits → finer staircase → smoother wave (slide 17: 3-bit vs 16-bit).

9 • All five waves at a glance (comparison)

Wave	P1 pattern over time	Made by	Period T ≈	Frequency set by
Square	`00H ↔ FFH`	2 levels + 2 EQUAL delays	2 × delay	the equal delays
Pulsed	`FFH(short), 00H(long)`	2 UNEQUAL delays	t_short + t_long	both delays
Triangular	`00H→FFH→00H ramps`	INC A ramp + DEC A ramp	2 × 255 × t_step	delay per step
Sawtooth	`00H→FFH, snap 0`	INC A ramp only (rollover)	255 × t_step	delay per step
Sine	`100 table samples`	ROM table + MOVC loop	100 × t_sample	delay + sample count

30-second revision: P1 value → DAC current → op-amp → Vout • SHAPE = number sequence, FREQUENCY = delay • square: 00/FF equal • pulsed: unequal • triangle: INC + DEC ramps • sawtooth: INC ramp + rollover • sine: DB table (128 + 127·sin), MOVC, restart after 100 samples.